

Section 11.3.2 - Bag-of-words models

December 17, 2025

1 Bag-of-words models

- How to encode word order?
- 1) Unigrams with binary encoding - discard order
 - 2) Bigrams with binary encoding - inject local word order information
 - 3) Bigrams with TF-IDF encoding - inject local word order information and concern word frequency

1.1 The IMDB movie reviews data (raw)

- Sentiment classification
- Balanced two classes
- Use the utility function: `text_dataset_from_directory()`

```
[1]: from tensorflow import keras

batch_size = 32

train_ds = keras.utils.text_dataset_from_directory(
    "aclImdb/train", batch_size=batch_size
)
val_ds = keras.utils.text_dataset_from_directory(
    "aclImdb/val", batch_size=batch_size
)
test_ds = keras.utils.text_dataset_from_directory(
    "aclImdb/test", batch_size=batch_size
)
```

Found 20000 files belonging to 2 classes.

Found 5000 files belonging to 2 classes.

Found 25000 files belonging to 2 classes.

Shape and dtypes of the first batch

```
[2]: for inputs, targets in train_ds:
    print("inputs.shape:", inputs.shape)
    print("inputs.dtype:", inputs.dtype)
    print("targets.shape:", targets.shape)
```

```

print("targets.dtype:", targets.dtype)
print("inputs[0]:", inputs[0])
print("targets[0]:", targets[0])
break

```

```

inputs.shape: (32,)
inputs.dtype: <dtype: 'string'>
targets.shape: (32,)
targets.dtype: <dtype: 'int32'>
inputs[0]: tf.Tensor(b'Richard Willaims is an animation god. He was hampered in
directing this film by the producer. The final product is a very uneven film
with a very convoluted story, but some amazing moments of animation (like Emery
Hawkins\' "Greedy"). Joe Raposo\'s repetitive music doesn\'t help either. It was
made in wide screen so the VHS doesn\'t show it in all it\'s glory, let\'s hope
for a letterboxed DVD someday. Still it\'s worth watching for some eye popping
animation.', shape=(), dtype=string)
targets[0]: tf.Tensor(1, shape=(), dtype=int32)

```

Model-building utility function

```

[3]: from tensorflow import keras
from tensorflow.keras import layers

def get_model(max_tokens=20000, hidden_dim=16):
    inputs = keras.Input(shape=(max_tokens,))
    x = layers.Dense(hidden_dim, activation="relu")(inputs)
    x = layers.Dropout(0.5)(x)
    outputs = layers.Dense(1, activation="sigmoid")(x)
    model = keras.Model(inputs, outputs)
    model.compile(optimizer="rmsprop",
                  loss="binary_crossentropy",
                  metrics=["accuracy"])
    return model

```

1.2 Unigrams with binary encoding

- TextVectorization layer: output_mode = “multi_hot”
- Use catch() to preprocess the data once during the first epoch
- A sample is a binary vector of length max_token
- Discard word order
- Ignore frequency/count, only presence/absence matters

```

[4]: from tensorflow.keras.layers import TextVectorization

text_vectorization = TextVectorization(
    max_tokens=20000,
    output_mode="multi_hot",
)

```

```

text_only_train_ds = train_ds.map(lambda x, y: x)
text_vectorization.adapt(text_only_train_ds)

binary_1gram_train_ds = train_ds.map(
    lambda x, y: (text_vectorization(x), y),
    num_parallel_calls=4)
binary_1gram_val_ds = val_ds.map(
    lambda x, y: (text_vectorization(x), y),
    num_parallel_calls=4)
binary_1gram_test_ds = test_ds.map(
    lambda x, y: (text_vectorization(x), y),
    num_parallel_calls=4)

```

Samples of binary unigram dataset

```

[5]: for inputs, targets in binary_1gram_train_ds:
    print("inputs.shape:", inputs.shape)
    print("inputs.dtype:", inputs.dtype)
    print("targets.shape:", targets.shape)
    print("targets.dtype:", targets.dtype)
    print("inputs[0]:", inputs[0])
    print("targets[0]:", targets[0])
    break

```

```

inputs.shape: (32, 20000)
inputs.dtype: <dtype: 'float32'>
targets.shape: (32,)
targets.dtype: <dtype: 'int32'>
inputs[0]: tf.Tensor([1. 1. 1. ... 0. 0. 0.], shape=(20000,), dtype=float32)
targets[0]: tf.Tensor(1, shape=(), dtype=int32)

```

Building a model

```

[6]: from tensorflow import keras
    from tensorflow.keras import layers

    def get_model(max_tokens=20000, hidden_dim=16):
        inputs = keras.Input(shape=(max_tokens,))
        x = layers.Dense(hidden_dim, activation="relu")(inputs)
        x = layers.Dropout(0.5)(x)
        outputs = layers.Dense(1, activation="sigmoid")(x)
        model = keras.Model(inputs, outputs)
        model.compile(optimizer="rmsprop",
                      loss="binary_crossentropy",
                      metrics=["accuracy"])
        return model

```

Training and testing the model

```
[7]: model = get_model()
model.summary()

callbacks = [
    keras.callbacks.ModelCheckpoint("binary_1gram.h5",
                                    save_best_only=True)
]

history = model.fit(binary_1gram_train_ds.cache(),
                    validation_data=binary_1gram_val_ds.cache(),
                    epochs=10,
                    callbacks=callbacks)

model = keras.models.load_model("binary_1gram.h5")
print(f"Test acc: {model.evaluate(binary_1gram_test_ds)[1]:.3f}")
```

Model: "model"

Layer (type)	Output Shape	Param #
input_1 (InputLayer)	[(None, 20000)]	0
dense (Dense)	(None, 16)	320016
dropout (Dropout)	(None, 16)	0
dense_1 (Dense)	(None, 1)	17

```
=====  
Total params: 320,033  
Trainable params: 320,033  
Non-trainable params: 0
```

```
-----  
Epoch 1/10  
625/625 [=====] - 2s 3ms/step - loss: 0.4090 -  
accuracy: 0.8260 - val_loss: 0.2927 - val_accuracy: 0.8798  
Epoch 2/10  
625/625 [=====] - 2s 3ms/step - loss: 0.2744 -  
accuracy: 0.8994 - val_loss: 0.2835 - val_accuracy: 0.8898  
Epoch 3/10  
625/625 [=====] - 2s 2ms/step - loss: 0.2458 -  
accuracy: 0.9144 - val_loss: 0.2917 - val_accuracy: 0.8928  
Epoch 4/10  
625/625 [=====] - 1s 2ms/step - loss: 0.2251 -  
accuracy: 0.9244 - val_loss: 0.3282 - val_accuracy: 0.8826  
Epoch 5/10  
625/625 [=====] - 1s 2ms/step - loss: 0.2174 -  
accuracy: 0.9296 - val_loss: 0.3207 - val_accuracy: 0.8896
```

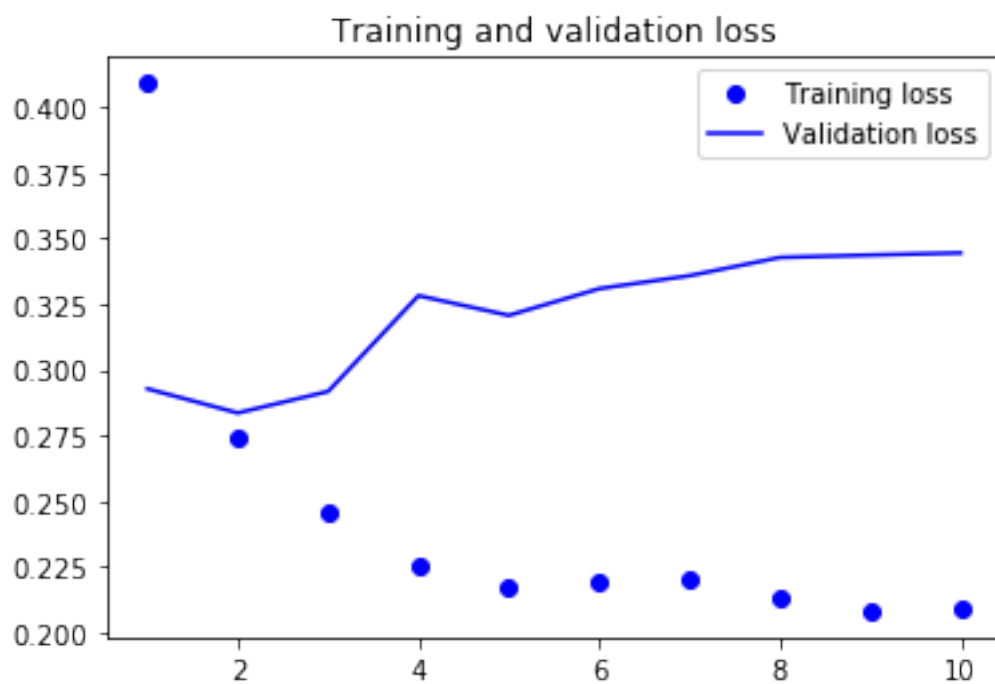
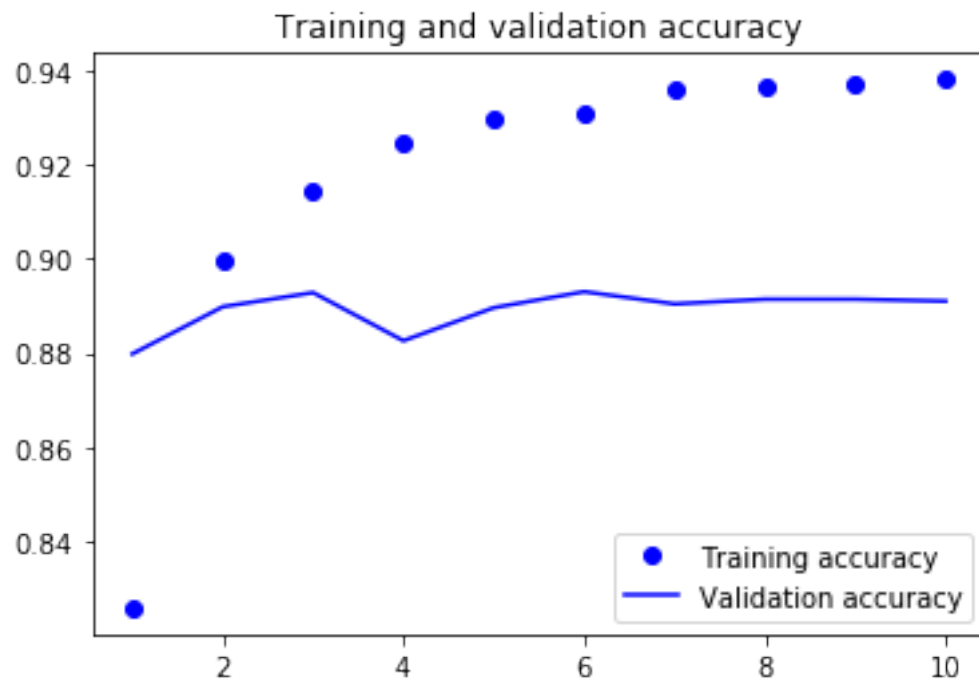
Epoch 6/10
 625/625 [=====] - 1s 2ms/step - loss: 0.2197 - accuracy: 0.9305 - val_loss: 0.3308 - val_accuracy: 0.8930
 Epoch 7/10
 625/625 [=====] - 2s 2ms/step - loss: 0.2198 - accuracy: 0.9361 - val_loss: 0.3358 - val_accuracy: 0.8904
 Epoch 8/10
 625/625 [=====] - 1s 2ms/step - loss: 0.2133 - accuracy: 0.9366 - val_loss: 0.3427 - val_accuracy: 0.8914
 Epoch 9/10
 625/625 [=====] - 1s 2ms/step - loss: 0.2082 - accuracy: 0.9370 - val_loss: 0.3436 - val_accuracy: 0.8914
 Epoch 10/10
 625/625 [=====] - 1s 2ms/step - loss: 0.2087 - accuracy: 0.9380 - val_loss: 0.3444 - val_accuracy: 0.8910
 782/782 [=====] - 1s 2ms/step - loss: 0.2996 - accuracy: 0.8803
 Test acc: 0.880

Plotting the results

- Test accuracy: 88%

```
[8]: import matplotlib.pyplot as plt

acc = history.history["accuracy"]
val_acc = history.history["val_accuracy"]
loss = history.history["loss"]
val_loss = history.history["val_loss"]
epochs = range(1, len(acc) + 1)
plt.plot(epochs, acc, "bo", label="Training accuracy")
plt.plot(epochs, val_acc, "b", label="Validation accuracy")
plt.title("Training and validation accuracy")
plt.legend()
plt.figure()
plt.plot(epochs, loss, "bo", label="Training loss")
plt.plot(epochs, val_loss, "b", label="Validation loss")
plt.title("Training and validation loss")
plt.legend()
plt.show()
```



Trained model incorporating text processing

```
[9]: inputs = keras.Input(shape=(1,), dtype="string")
processed_inputs = text_vectorization(inputs)
outputs = model(processed_inputs)
inference_model = keras.Model(inputs, outputs)
```

Predict on a test example

```
[10]: import tensorflow as tf
raw_text_data = tf.convert_to_tensor([
    ["That was an excellent movie, I loved it."],
])
predictions = inference_model(raw_text_data)
print(f"{float(predictions[0] * 100):.2f} percent positive")
```

80.83 percent positive

1.3 Bigrams with binary encoding

- TextVectorization layer: output_mode = “multi_hot”
- Use catch() to preprocess the data once during the first epoch
- A sample is a binary vector of length max_token
- Bigrams (including unigrams?): inject local word order information
- Different list of tokens than unigrams!
- Ignore frequency/count, only presence/absence matters

```
[11]: text_vectorization = TextVectorization(
    ngrams=2,
    max_tokens=20000,
    output_mode="multi_hot",
)

text_vectorization.adapt(text_only_train_ds)
binary_2gram_train_ds = train_ds.map(
    lambda x, y: (text_vectorization(x), y),
    num_parallel_calls=4)
binary_2gram_val_ds = val_ds.map(
    lambda x, y: (text_vectorization(x), y),
    num_parallel_calls=4)
binary_2gram_test_ds = test_ds.map(
    lambda x, y: (text_vectorization(x), y),
    num_parallel_calls=4)
```

Samples of binary bigram dataset

```
[12]: for inputs, targets in binary_2gram_train_ds:
    print("inputs.shape:", inputs.shape)
    print("inputs.dtype:", inputs.dtype)
    print("targets.shape:", targets.shape)
    print("targets.dtype:", targets.dtype)
```

```

print("inputs[0]:", inputs[0])
print("targets[0]:", targets[0])
break

```

```

inputs.shape: (32, 20000)
inputs.dtype: <dtype: 'float32'>
targets.shape: (32,)
targets.dtype: <dtype: 'int32'>
inputs[0]: tf.Tensor([1. 1. 1. ... 0. 0. 0.], shape=(20000,), dtype=float32)
targets[0]: tf.Tensor(1, shape=(), dtype=int32)

```

Building, training and testing the model

```

[13]: model = get_model()
      model.summary()

      callbacks = [
          keras.callbacks.ModelCheckpoint("binary_2gram.h5",
                                          save_best_only=True)
      ]

      history = model.fit(binary_2gram_train_ds.cache(),
                          validation_data=binary_2gram_val_ds.cache(),
                          epochs=10,
                          callbacks=callbacks)

      model = keras.models.load_model("binary_2gram.h5")
      print(f"Test acc: {model.evaluate(binary_2gram_test_ds)[1]:.3f}")

```

Model: "model_2"

Layer (type)	Output Shape	Param #
input_3 (InputLayer)	[(None, 20000)]	0
dense_2 (Dense)	(None, 16)	320016
dropout_1 (Dropout)	(None, 16)	0
dense_3 (Dense)	(None, 1)	17

```

=====
Total params: 320,033
Trainable params: 320,033
Non-trainable params: 0

```

```

-----
Epoch 1/10
625/625 [=====] - 2s 3ms/step - loss: 0.3776 -
accuracy: 0.8428 - val_loss: 0.2650 - val_accuracy: 0.8976

```



```

Epoch 2/10
625/625 [=====] - 2s 2ms/step - loss: 0.2401 -
accuracy: 0.9171 - val_loss: 0.2727 - val_accuracy: 0.8998
Epoch 3/10
625/625 [=====] - 2s 2ms/step - loss: 0.2018 -
accuracy: 0.9352 - val_loss: 0.3010 - val_accuracy: 0.8994
Epoch 4/10
625/625 [=====] - 2s 3ms/step - loss: 0.1939 -
accuracy: 0.9420 - val_loss: 0.3170 - val_accuracy: 0.9000
Epoch 5/10
625/625 [=====] - 1s 2ms/step - loss: 0.1802 -
accuracy: 0.9477 - val_loss: 0.3295 - val_accuracy: 0.8980
Epoch 6/10
625/625 [=====] - 2s 2ms/step - loss: 0.1758 -
accuracy: 0.9483 - val_loss: 0.3417 - val_accuracy: 0.8976
Epoch 7/10
625/625 [=====] - 1s 2ms/step - loss: 0.1747 -
accuracy: 0.9510 - val_loss: 0.3639 - val_accuracy: 0.8986
Epoch 8/10
625/625 [=====] - 1s 2ms/step - loss: 0.1757 -
accuracy: 0.9519 - val_loss: 0.3762 - val_accuracy: 0.8980
Epoch 9/10
625/625 [=====] - 1s 2ms/step - loss: 0.1721 -
accuracy: 0.9534 - val_loss: 0.3765 - val_accuracy: 0.8958
Epoch 10/10
625/625 [=====] - 1s 2ms/step - loss: 0.1787 -
accuracy: 0.9538 - val_loss: 0.3912 - val_accuracy: 0.8960
782/782 [=====] - 1s 2ms/step - loss: 0.2640 -
accuracy: 0.8986
Test acc: 0.899

```

Plotting the results

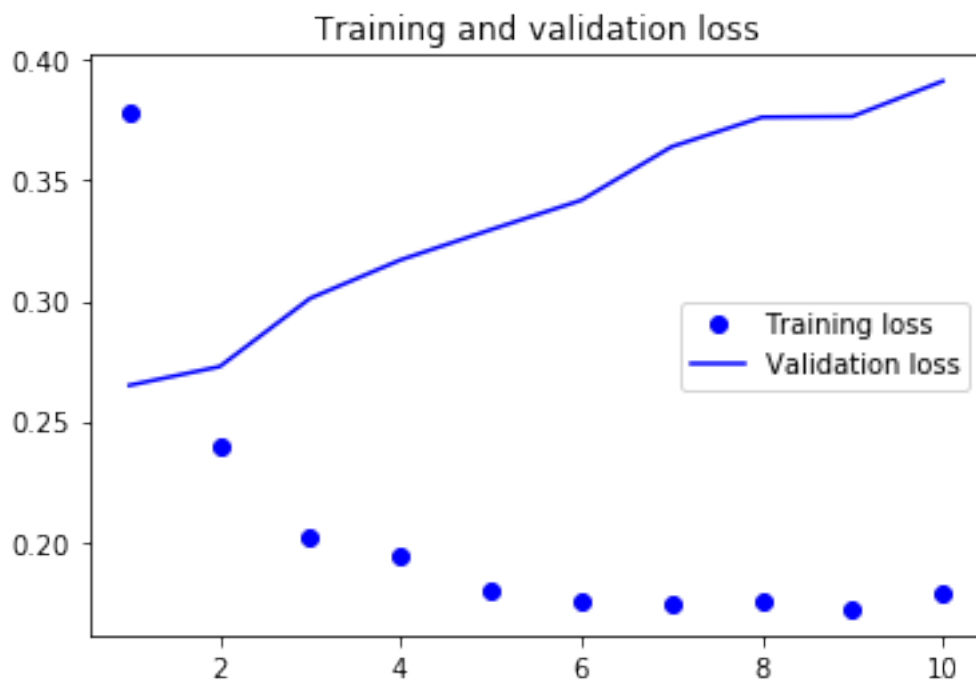
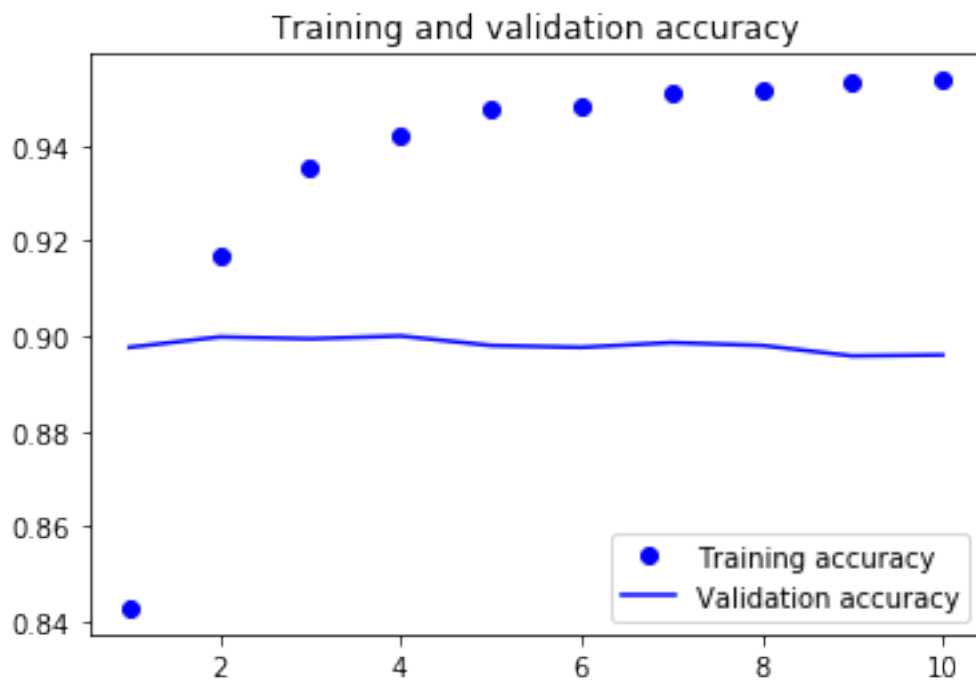
- Test accuracy: 90%, compared to 88% of the unigrams with binary encoding model.

```

[14]: acc = history.history["accuracy"]
      val_acc = history.history["val_accuracy"]
      loss = history.history["loss"]
      val_loss = history.history["val_loss"]
      epochs = range(1, len(acc) + 1)
      plt.plot(epochs, acc, "bo", label="Training accuracy")
      plt.plot(epochs, val_acc, "b", label="Validation accuracy")
      plt.title("Training and validation accuracy")
      plt.legend()
      plt.figure()
      plt.plot(epochs, loss, "bo", label="Training loss")
      plt.plot(epochs, val_loss, "b", label="Validation loss")
      plt.title("Training and validation loss")

```

```
plt.legend()  
plt.show()
```



Trained model incorporating text processing

```
[15]: inputs = keras.Input(shape=(1,), dtype="string")
      processed_inputs = text_vectorization(inputs)
      outputs = model(processed_inputs)
      inference_model = keras.Model(inputs, outputs)
```

Predict on a test example

```
[16]: raw_text_data = tf.convert_to_tensor([
      ["That was an excellent movie, I loved it."],
      ])
      predictions = inference_model(raw_text_data)
      print(f"{float(predictions[0] * 100):.2f} percent positive")
```

79.08 percent positive

1.4 Bigrams with TF-IDF encoding

- TextVectorization layer: output_mode = "tf_idf"
- Use catch() to preprocess the data once during the first epoch
- A sample is a float-valued vector of length max_token
- Bigrams inject local word order information
- Different list of tokens than unigrams!
- Reward more frequent words in the sample (frequency)
- Softly penalize word are more common across training samples (not the corpus frequency!)

```
[17]: text_vectorization = TextVectorization(
      ngrams=2,
      max_tokens=20000,
      output_mode="tf_idf",
      )

      text_only_train_ds = train_ds.map(lambda x, y: x)
      text_vectorization.adapt(text_only_train_ds)

      tfidf_2gram_train_ds = train_ds.map(
          lambda x, y: (text_vectorization(x), y),
          num_parallel_calls=4)
      tfidf_2gram_val_ds = val_ds.map(
          lambda x, y: (text_vectorization(x), y),
          num_parallel_calls=4)
      tfidf_2gram_test_ds = test_ds.map(
          lambda x, y: (text_vectorization(x), y),
          num_parallel_calls=4)
```

Samples of bigrams with TF-IDF encoding dataset

```
[18]: for inputs, targets in tfidf_2gram_train_ds:
        print("inputs.shape:", inputs.shape)
        print("inputs.dtype:", inputs.dtype)
        print("targets.shape:", targets.shape)
        print("targets.dtype:", targets.dtype)
        print("inputs[0]:", inputs[0])
        print("targets[0]:", targets[0])
        break
```

```
inputs.shape: (32, 20000)
inputs.dtype: <dtype: 'float32'>
targets.shape: (32,)
targets.dtype: <dtype: 'int32'>
inputs[0]: tf.Tensor(
[341.25168      4.8821487   3.5582361 ...   0.          0.
    0.          ], shape=(20000,), dtype=float32)
targets[0]: tf.Tensor(1, shape=(), dtype=int32)
```

Building, training and testing the model

```
[19]: model = get_model()
model.summary()

callbacks = [
    keras.callbacks.ModelCheckpoint("tfidf_2gram.h5",
                                    save_best_only=True)
]

history = model.fit(tfidf_2gram_train_ds.cache(),
                    validation_data=tfidf_2gram_val_ds.cache(),
                    epochs=10,
                    callbacks=callbacks)

model = keras.models.load_model("tfidf_2gram.h5")
print(f"Test acc: {model.evaluate(tfidf_2gram_test_ds)[1]:.3f}")
```

Model: "model_4"

Layer (type)	Output Shape	Param #
input_5 (InputLayer)	[(None, 20000)]	0
dense_4 (Dense)	(None, 16)	320016
dropout_2 (Dropout)	(None, 16)	0
dense_5 (Dense)	(None, 1)	17

Total params: 320,033
Trainable params: 320,033
Non-trainable params: 0

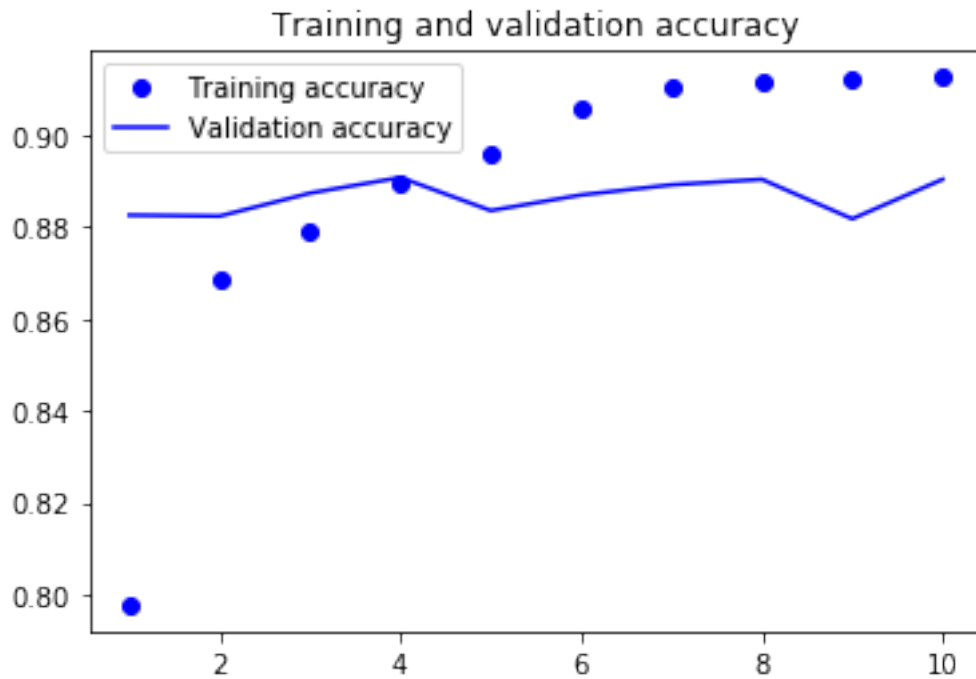
```
-----  
Epoch 1/10  
625/625 [=====] - 2s 3ms/step - loss: 0.5149 -  
accuracy: 0.7977 - val_loss: 0.3150 - val_accuracy: 0.8826  
Epoch 2/10  
625/625 [=====] - 1s 2ms/step - loss: 0.3171 -  
accuracy: 0.8686 - val_loss: 0.3254 - val_accuracy: 0.8824  
Epoch 3/10  
625/625 [=====] - 1s 2ms/step - loss: 0.2860 -  
accuracy: 0.8793 - val_loss: 0.3789 - val_accuracy: 0.8874  
Epoch 4/10  
625/625 [=====] - 1s 2ms/step - loss: 0.2635 -  
accuracy: 0.8897 - val_loss: 0.3558 - val_accuracy: 0.8908  
Epoch 5/10  
625/625 [=====] - 2s 2ms/step - loss: 0.2496 -  
accuracy: 0.8958 - val_loss: 0.3682 - val_accuracy: 0.8836  
Epoch 6/10  
625/625 [=====] - 2s 2ms/step - loss: 0.2394 -  
accuracy: 0.9060 - val_loss: 0.3739 - val_accuracy: 0.8870  
Epoch 7/10  
625/625 [=====] - 1s 2ms/step - loss: 0.2277 -  
accuracy: 0.9103 - val_loss: 0.3588 - val_accuracy: 0.8892  
Epoch 8/10  
625/625 [=====] - 1s 2ms/step - loss: 0.2182 -  
accuracy: 0.9118 - val_loss: 0.3844 - val_accuracy: 0.8904  
Epoch 9/10  
625/625 [=====] - 1s 2ms/step - loss: 0.2229 -  
accuracy: 0.9124 - val_loss: 0.4021 - val_accuracy: 0.8818  
Epoch 10/10  
625/625 [=====] - 1s 2ms/step - loss: 0.2137 -  
accuracy: 0.9126 - val_loss: 0.3900 - val_accuracy: 0.8904  
782/782 [=====] - 1s 2ms/step - loss: 0.3173 -  
accuracy: 0.8808  
Test acc: 0.881
```

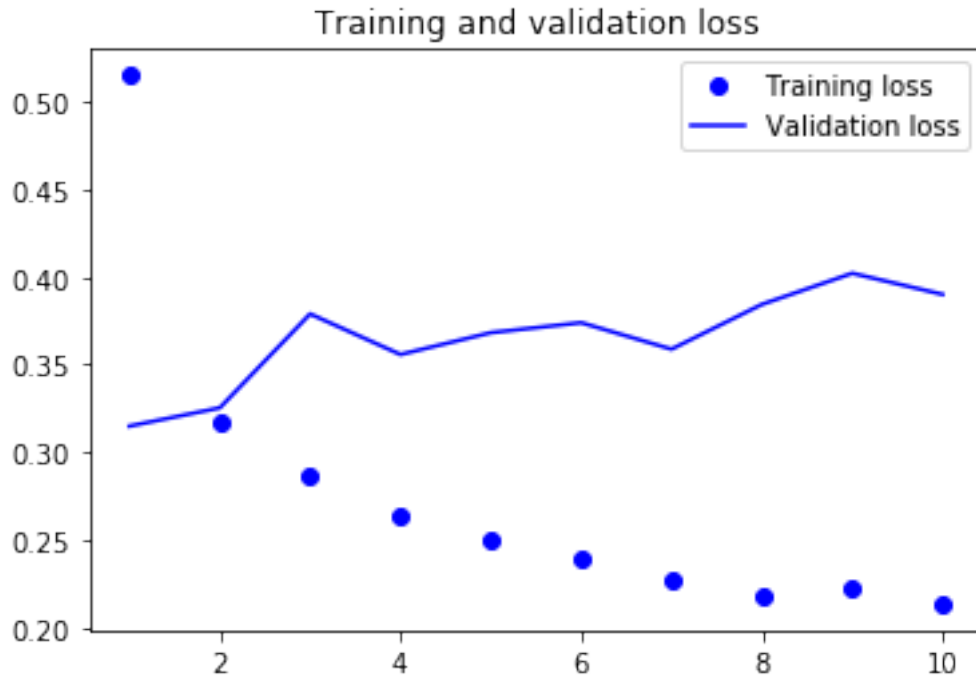
Plotting the results

- Test accuracy: 88.1%, compared to 88% (resp. 90%) of the unigrams (resp. bigrams) with binary encoding model.

```
[20]: acc = history.history["accuracy"]  
      val_acc = history.history["val_accuracy"]  
      loss = history.history["loss"]  
      val_loss = history.history["val_loss"]  
      epochs = range(1, len(acc) + 1)
```

```
plt.plot(epochs, acc, "bo", label="Training accuracy")
plt.plot(epochs, val_acc, "b", label="Validation accuracy")
plt.title("Training and validation accuracy")
plt.legend()
plt.figure()
plt.plot(epochs, loss, "bo", label="Training loss")
plt.plot(epochs, val_loss, "b", label="Validation loss")
plt.title("Training and validation loss")
plt.legend()
plt.show()
```





Trained model incorporating text processing

```
[21]: inputs = keras.Input(shape=(1,), dtype="string")
      processed_inputs = text_vectorization(inputs)
      outputs = model(processed_inputs)
      inference_model = keras.Model(inputs, outputs)
```

Predict on a test example

```
[22]: raw_text_data = tf.convert_to_tensor([
      ["That was an excellent movie, I loved it."],
    ])
      predictions = inference_model(raw_text_data)
      print(f"{float(predictions[0] * 100):.2f} percent positive")
```

88.92 percent positive